Project 2: Multi-ECU automotive CAN network

May 25, 2026 4:08 PM

What it is:

- It will be simulated, to make it seem like im working with real vehicle data. We create TWO independent nodes on one board. CAN0 acts as an Engine ECU broadcasting RMP, throttle, speed, and temp at realistic automotive message rates. CAN1 acts as a 'Body Control Module' that listens and responds. Then comes a third firmware layer. A lightweight intrusion detection system running on the same core - it monitors bus traffic for anomalies: timing violations, unexpected message IDs, value spoofing, etc. Finally, a python dashboard on my laptop will visualise the live traffic and flag anomalies in real-time. This is pure automotive engineering without access to a car - my initial thought was hooking up my laptop to the car using an obd-II port, but couldn't find a car.
- Setup: MSP432 CAN0 -> Transceiver -> CAN Bus wires -> Transceiver -> MSP432 CAN1

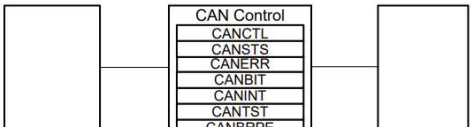
Notes from phases and initial research (reading theory):

Let's read from the TRM's section on CAN networks

- Controller Area Network (CAN) is a multicast, shared serial bus standard for connecting electronic control units (ECUs). It is robust in electromagnetically-noisy environments and can use differential balanced line like RS-485 or a more robust twisted-pair wire. We will use two basic wires, twist them into a pair to create the CAN_H and CAN_L bus.
- Bits rates of up to 1Mbps are possible at lengths less than 40m. Decreased bit rates allow for an increased bus length.
- Our ARM cortex mcu has two CAN units, protocol version 2.0 part A/B
- Bit rates of up to 1Mbps. 32 message objects per controller.
- Attaches to an external CAN transceiver through the CAN_TX and CAN_RX signals.
- I have ordered 3 SN65HVD230 CAN transceivers and will use them to create the bus.
- Block diagram can be seen below

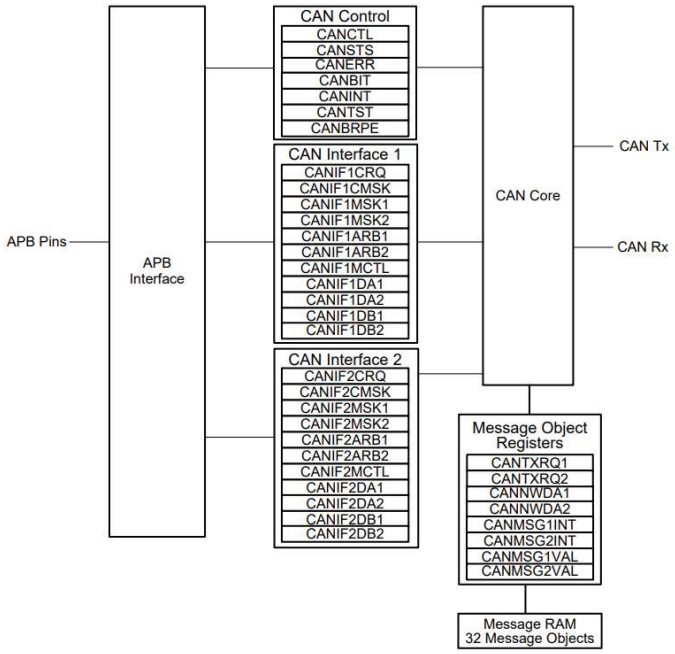
11.2 Block Diagram

Figure 11-1 shows the CAN Controller block diagram.



11.2 Block Diagram

Figure 11-1 shows the CAN Controller block diagram.



-

- Messages split into ID and payload. CAN ID 1 has the highest priority on the system.

A data frame contains data for transmission, whereas a remote frame contains no data and is used to request the transmission of a specific message object. The CAN data frame or remote frame is constructed as shown in Figure 11-2.

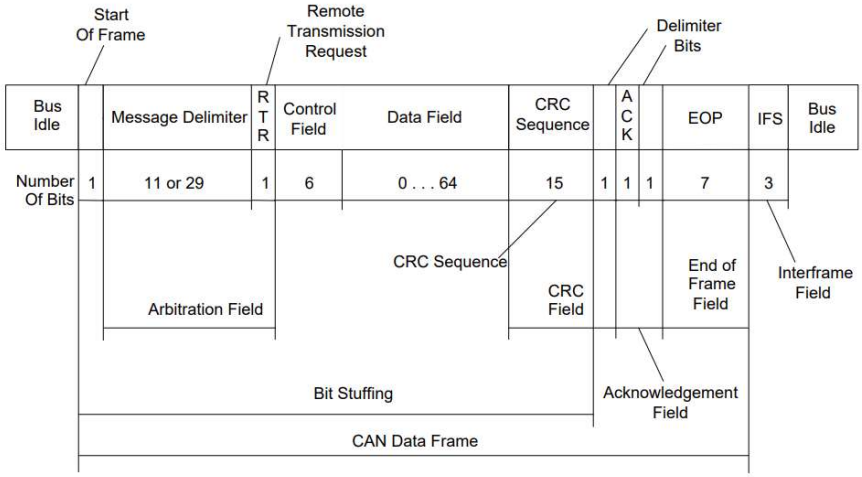


Figure 11-2. CAN Data Frame or Remote Frame

-

- Let's understand this better before moving on
- Start of Frame (SOF): synchronise clocks
- The Arbitration Field (ID): It is an 11-bit (in our case, as we don't want the extended 29-bit version) Message Identifier. This ID says WHAT the data is.
- Lower IDs have higher priorities
- Priorities matter in CAN when two nodes try to talk at the same time.

- RTR (remote transmission request): usually 0. if it is a 1, it means the node is asking the other node to send data rather than sending it itself.
- Control Field: 6-bit section that tells the receiver how many bytes of data are about to follow.
- Data Field: Actual payload - 0 to 8 bytes of raw data
- CRC Sequence: 15-bit checksum
- ACK: transmitting node sends a logic 1 bit. If the receiving node successfully received the message without errors, it will pull this bit low to 0, therefore acknowledging.
- EOF: Resets the frame

- A very important aspect of this is the 32 message objects which are 32 dedicated hardware mailboxes built directly into the CAN controller SRAM.
- We can configure our message objects (mailboxes) to only accept certain things. This can be used for the IDS im thinking of building on top of this.

Init Sequence:

- To use CAN on this board, the peripheral clock must be enabled using RCGCCAN register.
- Additionally, the clock to the GPIO must be enabled using the RCGCGPIO register.
- We need GPIO port A and B for the CAN network
- PA0 and PA1 -> CAN0Rx and CAN0Tx
- PB0 and PB1 -> CAN1Rx and CAN1Tx
- Finally found the holes in the mcu for this
- So, there are no normal headers for these pins - so I bent the wires 90 degrees at the end to make them stick in the holes
- Another thing - looking carefully at the board, I was able to figure out HOW TO ENABLE PA0 and PA1. It was to rotate the connecter on the two JP5 and JP4 sections of the board 90 degrees.
- Set the GPIO AFSEL bits for the appropriate pins
- Configure the PMCn fields in the GPIOPCTL register to assign the CAN signals to the appropriate pins.

- Software initialisation includes setting the INIT bit in the CANCTL register. When INIT is set, all message transfers to and from the CAN bus are stopped and CANnTX signal is held high.

- To initialise the CAN controller, set the CAN Bit Timing (CANBIT) register and configure each message object. If a message object is not needed, label it as not valid by clearing the MSGVAL bit in the

CAN IFn Arbitration 2 register.

- Both the INIT and CCE bits in the CANCTRL register must be set in order to access the CANBIT register and the CAN Baude Rate Prescaler Extension (CANBRPE) register to configure the bit timing. The init must be cleared to leave the initialisation stage.

- Now, let's discuss Bit time and Bit rate. A single CAN bit is not just one clock pulse. It is a slice of time divided into segments, and we control how wide each segment is. This is how every node on the bus stays synchronised even with slightly different crystal oscillators
- We have 4 segments to work with:
  a. Sync_Seg -> always exactly 1 time quantum. Fixed.
  b. Prop_Seg2 + Phase_Seg1 -> Texas Instruments combines these into one field called TSEG1 in the CANBIT register. This segment is where the bus signal stabilizes and propagates. The sample point which is the exact moment the controller reads the bit value falls at the end of TSEG1.
  c. Phase_Seg2 -> Called TSEG2 in CANBIT. After some sample point. The controller can shorten this segment to re-sync if it detects a late edge. This is what gives CAN its noise tolerance.
  d. SJW (synchronisation jump width) -> Not a segment, but a parameter. It defines the maximum number of time quanta the controller is allowed to add or subtract from TSEG1.TSEG2 to resynchronize. Usually set to 1 or 2.

- Formula: Bit Rate = System_Clock / ((BRP + 1) * (1 + TSEG1 + TSEG2))
- BRP -> Baud Rate Prescaler. Hardware component that divides a high-frequency system clock to a slower, specific frequency. It allows the device to generate the exact, standardized communication speeds required for CAN.
- A time quanta (or quantum - something the datasheet refers to a lot) is the Fixed, maximum amount of CPU time allocated to a process in a multitasking or timesharing operating system.
- Ok so we know that our system clock is 120MHz. Our goal bit rate is 500kbp (standard). TQ = 1 + TSEG1 + TSEG2. Our TQ is 16 (industry standard). Therefore, solving the above formula, we get BRP to be 14. Sample point must be around 80%. Setting TSEG1=12 and TSEG2=3 gives you a sample point of 81.25%. SJW = 1
- Remember that CANBIT stores TSEG as value - 1. The hardware adds 1 back internally.
- So, for the code, BRP=13, TSEG1=11, TSEG2=2, SJW=0

- Message Object interface: the 32 message objects live in SRAM in the CAN controller. This means that the CAN must constantly access the SRAM in hardware. Might end up in a race condition with some other cpu task. We use an indirect two-step interface. We stage everything into the IF (interface) register first. They are like staging buffers.
- Then we write the message object number to the COMMAND REQUEST register and set direction bits in the COMMAND MASK register. This triggers a transfer between the Interface (IF) staging buffer and the actual message object. NO collision
- IF1 and IF2 are two independent sets of registers. While IF1's transfer is still completing on one message object, you're already loading IF2 for the next one
- Sequence:
    - Load IF Command mask - specify what parts we are writing
    - Load IF arbitration registers - CAN ID, TX vs RX, MSGVAL bit
    - Load IF Message Control - data length, interrupt enable, TX request
    - Load IF Data Registers - your payload bytes (TX only)
    - Write object number to IF command request - this fires the transfer.

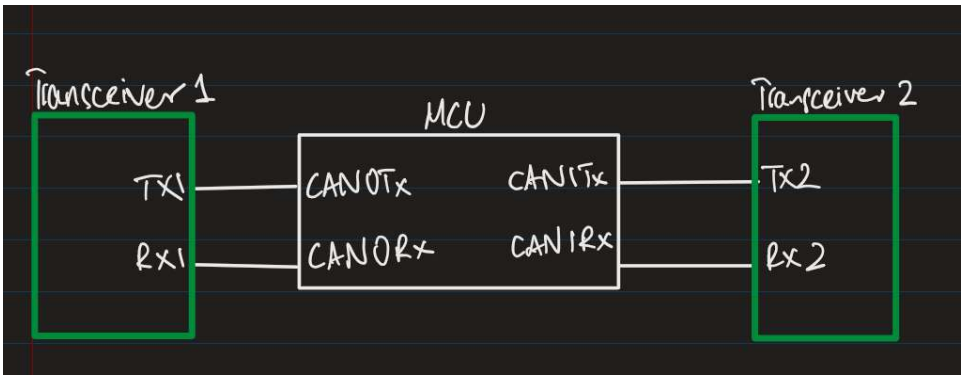
Hardware setup:

- We need termination resistors (2 x 120ohm) for the CAN network. They are used to match the characteristic impedance, Z, of the wire and prevent signal reflections. Ok, so we have built-in resistors in the transceivers. Did NOT need to find all those 240s
- MCU PA1 (CAN0TX, output) -> Transceiver TXD (input)
- MCU PA0 (CAN0RX, input) <- Transceiver RXD (output)
- So basically the transceiver TXD is the input to the MCU's TX output. Similar set up for RX.

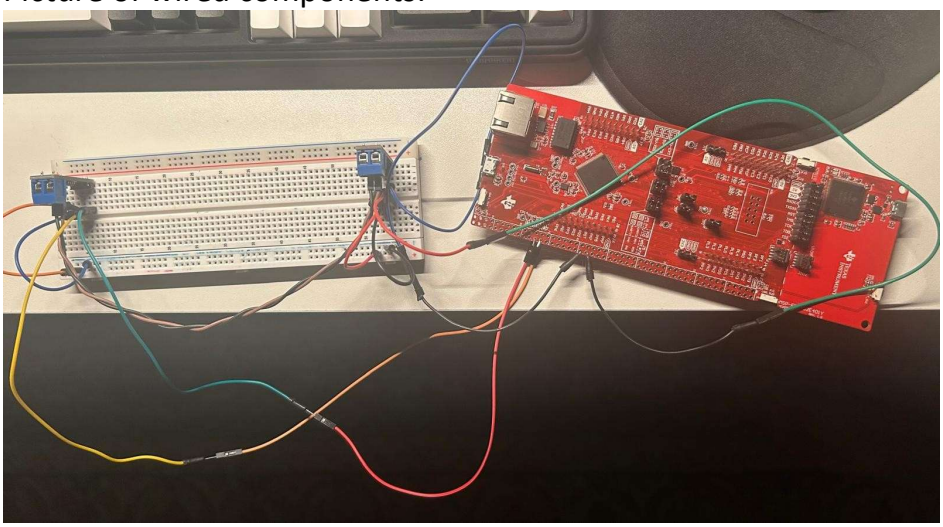
Wiring Setup:

- CANH on Transceiver 1 to CANH on Transceiver 2 - CANL lines do the same - both wires are twisted together for signal integrity (resolve the emi)
- Transceiver 1 -> VCC to 3.3V line, GND to common GND line, TXD to MCU's PA1 (CAN0TX), RXD to MCU's PA0 (CAN0RX)
- Transceiver 2 -> VCC to 3.3V line, GND to common GND line, TXD to PB1 and RXD to PB0

Design:



Picture of wired components:



Some new initialisation knowledge:

- AFSEL (Alternate Function Select): We hand control over to a hardware peripheral. We don't use standard GPIO data register for this pin
- PCTL (Port Control): Selects which peripheral gets the pin (e.g. routing CAN0Tx to this pin, not UART1 TX)
- DEN we know is digital enable. It physically turns on the buffers. Basically, the pin goes live. If we do DEN before setting the previous configurations, there would be a lot of chaos.
- GPIO port A: 0x40058000
  GPIO port B: 0x40059000
  RCGCAN Register Bits 0 and 1 if SET enable the clock to CAN module 0 and 1 in run mode respectively. Bits 2-31 are reserved
  To enable clock for the specific gpio registers, we use the RCGCGPIO register - Bit 0 and Bit 1 for PA and PB respectively.
  GPIOAFSEL register is used as a mode control register. if a bit is clear, the pin is used as a GPIO and is controlled by gpio pins. Else, it is controlled by an associated peripheral. the GPIO port control (GPIOPCTL) register is used to selecred on of the possible functions.
  Bits 0-7 -> 0x1 to be controlled by alternate hardware function.

- For the PRGPIO, I found that bits 0 and 1 are responsible for Port A and Port B. Setting the two bits means they are ready for access.
- PCTL values of 7 for both.

- Ok so remember one rule: Every init sequence starts with two clock enables; one for the peripheral itself (CAN in this case) and one for the GPIO port it uses (Port A/B in this case)

- Refer to the can.c file for the CAN setup - a lot of notes and values - setting it up was all over the place - but nicely explained in the code comments so use that please.
- One thing, when working with the CANBIT timing register, the 4 values we calculated are used like this:

Table 11-11. CANBIT Register Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R	0x0	
14-12	TSEG2	R/W	0x2	Time Segment after Sample Point. 0x00 to 0x07: The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. So, for example, the reset value of 0x2 means that 3 (2+1) bit time quanta are defined for Phase2 (see Figure 11-4). The bit time quanta is defined by the BRP field.
11-8	TSEG1	R/W	0x3	Time Segment Before Sample Point. 0x00 to 0x0F: The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. So, for example, the reset value of 0x3 means that 4 (3+1) bit time quanta are defined for Phase1 (see Figure 11-4). The bit time quanta is defined by the BRP field.
7-6	SJW	R/W	0x0	(Re)Synchronization Jump Width. 0x00 to 0x03: The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. During the start of frame (SOF), if the CAN controller detects a phase error (misalignment), it can adjust the length of TSEG2 or TSEG1 by the value in SJW. So the reset value of 0 adjusts the length by 1 bit time quanta.
5-0	BRP	R/W	0x1	Baud Rate Prescaler. The value by which the oscillator frequency is divided for generating the bit time quanta. The bit time is built up from a multiple of this quantum. 0x00 to 0x03F: The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. BRP defines the number of CAN clock periods that make up 1 bit time quanta, so the reset value is 2 bit time quanta (1+1). The CANBRPE register can be used to further divide the bit time.

-

- Ok ykw we can explain the process here now that I get it better for the other stuff as well. Just remember whatever we do for CAN0, we do that for CAN1 as well (just diff bits).
- First, we work with the RCGCCAN register - enable clock for CAN0 by setting bit 0. Then, poll until bit 0 is set for the PRCAN register. Also remember that R0 is Port A so that loops to CAN0.
- Set INIT bit and CCE bit to 1 in the CAN0_CTL_R register. Use the macros already defined in the files for setting the bit.

- Ok so now Im working on the CANBIT register and filling its value based on the BRP, SJW, TSEG1 and TSEG2 numbers we found from the formulas and required speed of 500kbps. For this, we use basic bit shifting and ORing them together to make the value for the register - We can see my working in the rough notes for Project 2.

- First step is to make sure everything is in order and works - so we will transmit a message from one to the other - for this, we need to have 1 TX object on CAN0 and 1 RX object on CAN1
- MSGVAL bit -> if 1, then it is active, else ignored.

- Ok so my work was all over the place - so here is the final polished version of what I did for the setup:
  Phase A — Peripheral bring-up (CAN0_Init and CAN1_Init, identical)
  - Enable CAN peripheral clock via RCGCCAN
  - Poll PRCAN until the controller's ready bit is set
  - Set INIT bit in CANCTL (stops bus activity, safe to configure)
  - Set CCE bit in CANCTL (unlocks CANBIT access)
  - Write CANBIT: BRP=14, SJW=0, TSEG1=11, TSEG2=2
  - Clear CCE bit
  - Clear INIT bit (controller now synchronizes and goes online)

  Phase B — RX object config (inside CAN1_Init, after Phase A)
  - Set WRNRD + ARB + CONTROL bits in CANIF2CMSK (writing, transfer arbitration + control)
  - Set ID filter in CANIF2ARB2 (ID << 2 for 11-bit), DIR=0 (receive), MSGVAL=1, XTD=0
  - In CANIF2MCTL: set DLC, set EOB, UMASK=0 (accept exact ID only for now)
  - Write object number to CANIF2CRQ to commit

  Phase C — TX object config + send (CAN0_Transmit)
  - Set WRNRD + ARB + CONTROL + DATAA/DATAB in CANIF1CMSK
  - Set ID in CANIF1ARB2 (ID << 2), DIR=1 (transmit), MSGVAL=1, XTD=0
  - In CANIF1MCTL: set DLC, set EOB, set TXRQST
  - Load payload into CANIF1DA1/DA2/DB1/DB2
  - Write object number to CANIF1CRQ to commit → frame transmits ***(AI Generated notes)***

- Remember one thing - we are using different arbitration registers for both CAN0 and CAN1 so that they don't overlap.
- For reference on how I set up the message objects, refer to section 11.3.4 in the TRM which explains the steps.

- Ok so im setting the DLC right now and learning more about it - DLC is the **data length code** and is a 4 bit field that tells the controller how many bytes of data to expect or transmit. We can set it to 8 for now. 8 bytes of data uint32_t transmitting and receiving. So for now, I'm thinking of setting 0x08 in the register to get 8 bytes for

the message.

Ok done!!!! Interview ready notes below:

CAN Bring-Up Pipeline

1. **Turn on the clocks.** Enable the CAN peripheral clock (RCGCCAN) and the GPIO port clocks (RCGCGPIO), then wait for each to report ready. Nothing works until clocks are on.
2. **Route the pins.** Tell PA0/PA1 and PB0/PB1 to act as CAN pins instead of plain GPIO (AFSEL → PCTL → DEN, in that order).
3. **Freeze the controller to configure it.** Set INIT to take the controller off the bus, set CCE to unlock the timing register.
4. **Set the speed.** Write the bit-timing register so the bus runs at 500 kbps (this is the BRP/TSEG math).
5. **Bring it online.** Clear CCE, then clear INIT — now the controller syncs to the bus and is live.
6. **Arm the receiver (CAN1).** Configure one message object as a mailbox: tell it the ID to listen for, mark it receive-direction, mark it valid. Now it's waiting to catch frames.
7. **Build and fire a frame (CAN0).** Configure one message object as transmit: set the ID, set the data length, mark it valid, load the payload bytes, and set the transmit-request bit.
8. **Commit.** Write the message object's number into the command-request register. That single write hands everything to the hardware, and the frame goes out on the bus.

Testing the first frame of message transfer: (from the debugger)

- In the debugger for this step, we don’t need a watch variable. Rather, we need to watch CAN controller status registers directly. They are based in the **Peripheral View** (System Viewer -> CAN0/CAN1).
- CAN0 (TX) -> CANSTS -> the TXOK bit is set when a frame is successfully transmitted
- CAN1 (RX) -> CANSTS -> the RXOK bit is set when a valid frame is received. CANNWDA1 (New Data Register) -> bit 0 should be SET, meaning message object 1 now holds fresh data.

Now we need to check 'What' was actually transferred over the line:

- Remember: the payload sits in the message object's internal SRAM. We cant see this in any memory window. The only way to see it is to look into the IF registers. For this, we will write a function which is the reverse of everything we did to set up the RX system.

Debugging Phase 1:

- So after finishing the configurations, I am looking into the debugger

for checking all register values. I ran the code until a breakpoint which is added at the end of the CAN0_Transmit function.

- CAN0: The STS (status) register shows 0. This should show 1 if the transmission was successful. The ERR register's TEC shows 0 which should show a rising number if the transmission FAILED. So kinda stuck in the middle here. This basically indicates that the controller isn't even attempting to transmit. Because if it transmits, and fails, TEC should not be 0. SOO, RX is basically dominant and puller LOW cuz of the rule ive written down below.
- REMEMBER: A CAN Transmitter cannot complete a frame alone. It requires at least one other node to ACK it. So, we can also look into CAN1 and see what the issue could be.
- The BIT, CRQ, MSK and other configured registers look perfectly fine and working as intended. Same with CAN1
- So now by the looks of it, my code should be fine. Also, these exact steps I followed for the code were written in the TRM so these shouldn’t be wrong essentially. But, let's run some tests.
- We can also check the TXRQ1 register and its single bit. It says 1 which means THERE IS A transmission request. Now we have brough the problem down to the RX having some issue. Let's look into it.
- Something new I learned is that I can loop the TX and RX of the same node together just to test if my firmware is correct.
    - For this, we need to update the CAN0_Init() function.
    - Before clearing CCE and INIT bits AND after setting the CANBIT register
    - Set the Test bit in the Control Register - this unlocks the test register which is read-only until we set it
    - Now, in the TEST register, enable LBACK (internal loopback) and SILENT (nothing leaks onto the real bus). Both together, fully internal and bus removed.
    - Leave the configuration window (CCE and INIT bits cleared)
    - Run the debugger and check the TXOK bit in the STS (status) register.
- This worked, checked the wiring, tested the pins, rewired, the system worked normally without loopback :)

- Ok now we are doing the Receive function. Let's do some research first. We know that our CAN1 is the receiving node. We are in the interface register 2. Now, we also know that the RXOK bit can be checked to see if a receival happened. BUT, in order to know that a NEW object came in, we have to check NEWDAT. So, we poll that in the CANNWDA1 register.
- Then, we have to clear the WRNRD in the CMSK register. This will

allow us to do the IF operation in reverse, which is READ

- Ok so I got a bit confused in terms of data Direction. Basically, once we clear WRNRD bit, we know that the direction is Receival. The section bits (ARB, CONTROL, etc.) only pick WHICH parts move, not direction. So don't go looking for direction based registers in the section bits as it will mostly be one bit setting the mood beforehand.
- So, with this bit cleared, everything is moved OUT of the objects and into the IF2.
- Ok so this makes more sense now. With the direction set, now we need to move the arbitration bits into the IF2. this will transfer the ID, DIR, XTD, MSGVAL.
- Then CONTROL
- Then DATAA and DATAB to get the actual data. This brings the data from the transceiver into this IF.
- Clear the CLRINTPND and NEWDAT bits.
- The data lands in the data registers.
- NEWDAT was already cleared to detect the next frame.

1. **Wait for data.** Poll NEWDAT for object 1 (CANNWDA1 bit 0). Set = a fresh frame is in the mailbox.
2. **Set up a READ in the Command Mask.** Clear WRNRD — this flips the transfer direction to "out of the object." Select what to pull out: CONTROL (status/DLC), DATAA + DATAB (payload), ARB (ID). Also set CLRINTPND and NEWDAT so the hardware acknowledges and clears the frame flags during the read.
3. **Trigger the transfer.** Write the object number to IF2CRQ. This copies the selected sections *out* of the message object into the IF2 registers.
4. **Wait for BUSY to clear.**
5. **Read the results.** Payload from IF2DA1/DA2/DB1/DB2 (two bytes each, low byte first), DLC/status from IF2MCTL, ID from IF2ARB2. Unpack into a buffer.
6. **You're re-armed.** NEWDAT was auto-cleared in step 2, so the function is ready to catch the next frame.

Core concept: WRNRD sets *direction* (0 = read out of object); the ARB/CONTROL/DATA bits only select *what* moves; received bytes land in the IF2 **data** registers, never MCTL.

- Ok so the receive function is good - let's test it.
- ALL GOOD -it works - let's do the UART2 setup now
- One thing I need to remember - the clock registers are named using RCGC (Run Mode Gating Control).

Mistakes I learned from:

- For wiring, I did the TX->RX way which is how UART works, not this. Look at hardware section in this doc to see how it is actually wired.

- I was messing up on writing to specific bits of a register (idk how even after so much work in this area) - so just a thumb rule: To write a value into a field at bits [High: Low], you shift the value left by Low - the position of the field's lowest bit.
- Ok so im at the point where Im extracting the message on the receiving side and need to fill the message payload field. So I was trying to decide which design choice to go with. I was trying to hardcode the message in each register in the payload register - that was wrong lol. So, I decided to fill in the payload array (uint8_t payload[8]) manually. First element will be the Low Byte of the register. (0x1122) low byte is 0x22 and high byte is 0x11. second element is the high byte of the register. This helps us extract the bits properly. I made the mistake of getting the HIGH Byte by just doing Reg & 0xFF00. LOOK AT SIZES PROPERLY - the element going into the payload is a byte - and when we do 0xFF00 we extract the HIGH byte, BUT because it is 8 bits, the high ones get truncated and we're left with low byte again. SOOOOO, we need to do: (REG >> 8) & 0xFF - this makes sure the low bytes get totally removed.
- I made another local/global mistake - it was fine for debugging, but actual products need a proper pipeline. Sooo, I was declaring an instance of the Msg struct in the local stack of the CAN1_Receive function. This struct will disappear AS SOON AS the function returns. Therefore, we need to get a pointer in - this will ensure that the struct actually changes on a global level.
- Some notes I had to pick up while writing the receive function:

  Pointers & passing a struct to be filled (the &message fix)

  What I did wrong: I wrote Msg* message; — a pointer, but pointing at *garbage* (never aimed at a real struct). Then in the call I wrote CAN1_Receive(Msg* message), which is *declaration* syntax (a type), not a *call* (a value). The compiler said "unexpected type name 'Msg'" because a call wants a value, not a type.

  Two separate errors bundled together: (1) no real struct existed for the pointer to write into, and (2) I passed a type where an argument belongs.

  The fix:

  - Declare a **real struct**: Msg message; — this allocates actual storage.
  - Pass its **address**: CAN1_Receive(&message); — & gives the location of the real struct.
  - Function takes a pointer: void CAN1_Receive(Msg *message).
  - Inside, write through the pointer with ->: message->canID = ...

  The concept — why a pointer, not pass-by-value: C passes everything by *value* — a copy. If I passed the whole struct by value, the function would fill in a *copy* and my changes would vanish on return. By passing the *address* (still a value — the address gets copied), the function reaches back through it and writes into my *original* struct in main. That's the standard C idiom for an "output parameter": the function fills in a struct the caller owns. So message lives in main, and CAN1_Receive populates it in place.

  Why no extern: extern is for sharing one global across files. I don't need a global at all — main owns message, and I just hand its address to whoever needs to fill it. Cleaner, and it sidesteps the uninitialized-pointer trap entirely.

- Ok so we continue with mistakes on here.

Back to notes from here:

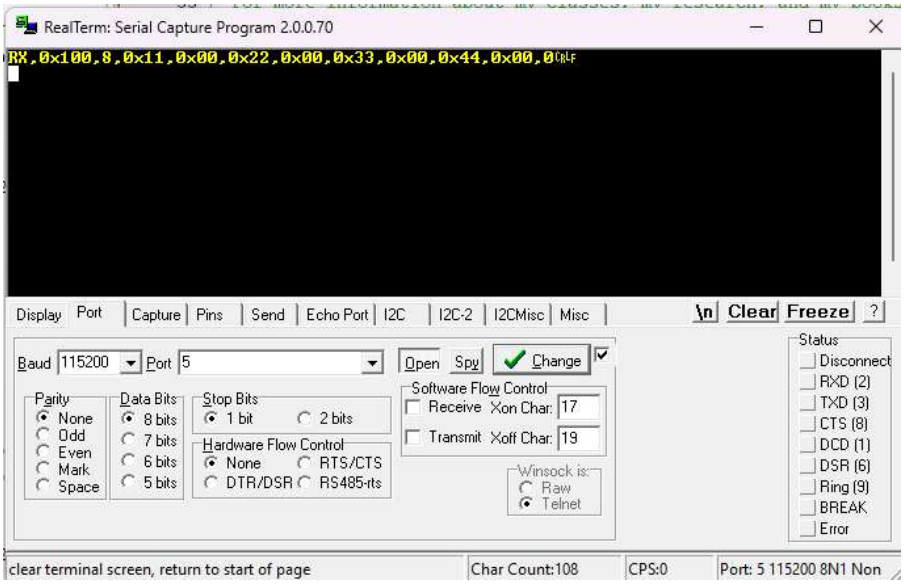
Result of CAN1_Receive:

Watch 1

Name	Value	Type
message	0x20000068 &message	struct <untagged>
canID	0x0100	ushort
DLC	0x08	uchar
payload	0x2000006B "□"	uchar[8]
[0]	0x11	uchar
[1]	0x00	uchar
[2]	0x22 '"'	uchar
[3]	0x00	uchar
[4]	0x33 '3'	uchar
[5]	0x00	uchar
[6]	0x44 'D'	uchar
[7]	0x00	uchar
overrunFlag	0x00000000	uint
timeStamp	0x00000000	uint
<Enter expression>		

DONEE - the message was successfully transmitted over the bus and saved into this struct.

- Ok so I did the UART stuff and it took me an entire day OMG - so I already turned the pin connectors on JP4 and JP5 90 degrees to enable both Port A0/A1 and UART2. I didn't see anything on realterm so I did the debugging test. Kept getting framing errors in the UART2 register. Turns out, I initially thought that the UART2 is internally traced and would not require a GPIO port. It in fact did need one and it ended up being PD4/PD5. as soon as I initialised that properly, UART2 came online and I could see the output on RealTerm
- Ok now im writing the firmware where we can output a uart trace which is both readable and parseable for the python program we're going to use later.
- So there must be a function we can call everytime we need a uart trace output. The function needs two things - a pointer to a msg struct (this will be pointing to the struct and not the data being copied itself everytime) and a direction. The pass by reference of the struct is needed to ensure that the whole data is copied itself.
- Ok so I made it so that there is the RX/TX,CANID,DLC,payload bits,timestamp. I used the bare-metal specific version of string printing talked about in the C syntax learned section.
- Result:

- 

- Ok doing the systick now - using the same systick from the rtos project (only init). The systick handler which runs every 1ms (configured in the systick_init), increments ticks by 1. in CAN1 _Receive, the UART trace also gets the timestamp which is just the ticks. It works!!!!

- Now we need to start on the 'simulation' - We want a data table with the ID, DLC, period, and payload. Period is the time before the next message is broadcasted.
- So we need a struct for each property. I was initially thinking of declaring a new Msg RPM, etc. property each time separately - but, a better approach is to use an array of structs. Message[0] can be RPM and so on - their identifications can be done using an enum.
- We can use a non-blocking delay for the periods ->
  Ticks - Last_transmitted >= Period
- Ok first step is to make a new function which populates all the different indices of the message struct
- Now im on the part where I need to recreate the CAN0_Transmit function in a way that it sets up the transmit for all message object in the Msg struct.
- Object Number != CANID -> for us, CANID is the differentiator, not for the hardware
- MNUM is the differentiator for the hardware.

- For the general steps, we can first create a function that sets up the registers once for the message transmission FOR EVERY EVENT. So we loop through all the indices in the message struct and do the following:
  - Set the WRNRD, ARB, and CONTROL bits of CMSK

    - Set the ID, MSGVAL, and DIR bits of ARB2 - enables transmission and is ready to be considered by the message handler
    - Set the DLC and EOB bits in the MCTL register
    - Create the message object using MNUM (the differentiator) -> CAN0_IF1CRQ_R = ((i + 1) << CAN_IF1CRQ_MNUM_S); //Message object i (1,2,3,4) - this creates the distinction in hardware
    - Then poll until the bus clears using the busy bit in the CRQ register and we're good

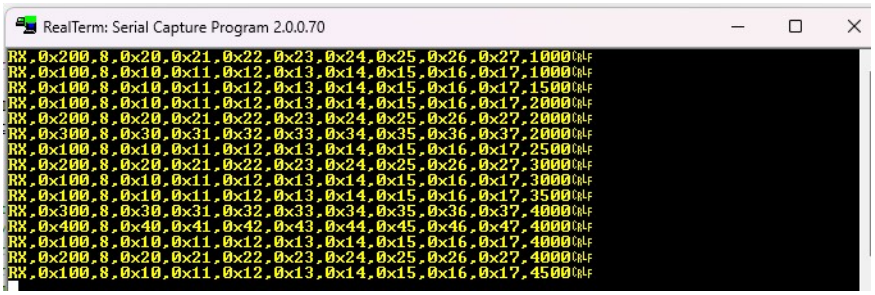
- Now, we need to setup the Transmission function that needs to happen EVERYtime a message transmits - not once per program - so this basically goes in the while(1) loop in main and under the function CAN0_Transmit - we only make one change where we add the index and a message struct pointer (pass-by-reference) as parameters.
- Now we only change the stuff we are left with after the TX_Setup. Set the WRNRD, DATAA, DATAB, and TXRQST. Fill in the payload registers using the payload arrays as well. Then, use MNUM for message object index and poll until BUSY clears.

- Now, let's do the continuous RX work. One thing we know is that we will use interrupt based RX. This is to make sure that the bus actually wakes up when a frame actually arrives. We never busy-wait.
- Ok so after some TRM research, I found that the MCTL (message control register) has a bit field called RXIE - Receive interrupt enable - if set, the INTPND bit in the same register is set after a successful reception of a frame. INTPND is the interrupt pending - if set, the message object becomes the source of the interrupt. The interrupt identifier in the CANINT register points to this message object if there is not another interrupt source with a higher priority.
- So we're working with the CANINT and MCTL registers - let's look more into how to make this work - paragraph from the TRM: If the system requires an interrupt on successful reception of a frame, the RXIE bit of the CANIFnMCTL register should be set. In this case, the INTPND bit of the same register is set, causing the CANINT register to point to the message object that just received a message. The TXRQST bit of this message object should be cleared to prevent the transmission of a remote frame. <> So this tells us on how to acc set up an interrupt based data frame reception.
- Ok so polling will be a problem - we are dealing with it by using interrupts. So we remove that from the code and keep the message object write.
- Now taking general notes from sections 11.3.7,11.3.10,11.3.12

- Ok so first thing would be to enable interrupts using the IE bit in the CANCTL register.
- Do the mctl stuff.
- If INTPND is set and a message arrives, the CANINT register automatically updates and points to the message object just received
- In the ISR, we clear INTPND
- After reading, clear NEWDAT and INTPND - indicates that the message has been received and read- onto the next
- If several interrupts are pending, the CAN Interrupt (CANINT) register points to the pending interrupt with the highest priority, disregarding their chronological order. The status interrupt has the highest priority. Among the message interrupts, the message object's interrupt with the lowest message number has the highest priority. A message interrupt is cleared by clearing the message object's INTPND bit in the CANIFnMCTL register or by reading the CAN Status (CANSTS) register. The status Interrupt is cleared by reading the CANSTS register. The interrupt identifier INTID in the CANINT register indicates the cause of the interrupt. When no interrupt is pending, the register reads as 0x0000. If the value of the INTID field is different from 0, then an interrupt is pending. If the IE bit is set in the CANCTL register, the interrupt line to the interrupt controller is active. The interrupt line remains active until the INTID field is 0, meaning that all interrupt sources have been cleared (the cause of the interrupt is reset), or until IE is cleared, which disables interrupts from the CAN controller. Functional Description www.ti.com 796 SLAU723A–October 2017–Revised October 2018 Submit Documentation Feedback Copyright © 2017–2018, Texas Instruments Incorporated Controller Area Network (CAN) Module The INTID field of the CANINT register points to the pending message interrupt with the highest interrupt priority. The SIE bit in the CANCTL register controls whether a change of the RXOK, TXOK, and LEC bits in the CANSTS register can cause an interrupt. The EIE bit in the CANCTL register controls whether a change of the BOFF and EWARN bits in the CANSTS register can cause an interrupt. The IE bit in the CANCTL register controls whether any interrupt from the CAN controller actually generates an interrupt to the interrupt controller. The CANINT register is updated even when the IE bit in the CANCTL register is clear, but the interrupt is not indicated to the CPU. A value of 0x8000 (INTID = 0x8000, which means the interrupt identifier is a status interrupt) in the CANINT register indicates that an interrupt is pending because the CAN module has updated (but not necessarily changed) the CANSTS register, indicating that either an error or status interrupt has been generated. A write access to the CANSTS register can clear the RXOK, TXOK, and LEC bits in that same register; however, the only way to clear the source of a status interrupt is to read the CANSTS register. The source of an interrupt can be determined in two ways during interrupt handling. The first is to read the INTID bit in the CANINT register to determine the highest priority interrupt that is pending, and the second is to read the CAN Message Interrupt Pending (CANMSGnINT) register to see all of the message objects that have pending interrupts. An interrupt service routine reading the message that is the source of the interrupt may read the message and clear the INTPND bit of the message object at the same time by setting the CLRINTPND bit in the CANIFnCMSK register. Once the INTPND bit has been cleared, the CANI

- So, it should follow similar structure as TX - working with a setup which is done once then a repetitive structure.
- Ok so what I did was remove the RX setup from the init function and moved it into a new function - the init function ends on clearing the CCE and INIT bits just like TX. This new function sets WRNRD, ARB, and CONTROL bits of CMSK. It sets the MSGVAL and inserts canID into CMSK. It also sets the RXIE, EOB, and Ors 0x08 with it for the MCTL. Then, it writes the object number (i + 1) and then polls using the busy bit. Then, sets bit 1 of CANCTL register which is the IE -Interrupt enable. Also, this all goes under a loop

using I of course
- Now, working with the CPU side of interrupts and NVIC and handlers and stuff
- So let's search up what the CAN_Handler is in my TRM
- So it is called CAN1_IRQHandler (interrupt request handler)

- So I initially thought that as soon as a message is sent, we are going into an interrupt, and waiting until receival is acked. BUT, the message is already received before the ISR ever runs. So, once the message has been transmitted, the receiving end register sets the RXIE bit in order to enable interrupts (something we are doing for all 4 events in our case when they're received). NOW, the controller raises an interrupt - the CPU pauses the main loop, runs the ISR, then returns.
- In the ISR, you acknowledge the interrupt by clearing the INTPND flag. Preempt -> ISR -> return
- So our CAN1_Receive must now become the ISR which also clears the INTPND flag.
- So we remove the NEWDAT Poll - we don't need it because the interrupt handles that time gap.
- Ok so something new - for this, we don't use MNUM - we use CANINT which already stored the index that was fired. That becomes the index.
- Success - 4 different messages being sent over the bus over different times
- 

- The timestamps, data, and IDs are all perfect - the design and code worked (might need to brush up on this part for interviews - one of the toughest ones to clear)
- Grab the code snippet from the can.c file under the CAN_Message_Table_Init(volatile Msg* msg) function which sets all these messages up.

- Now let's work on the IDS (intrusion detection system).
- IDS monitors traffic or system logs for malicious activity and alerts security teams. It doesn't block threats like a firewall.
- Key wording: It looks for the SIGNATURE of KNOWN attack types or detecting activity that deviates from normal
- So based on that - let's design signatures (first layer) -> RPM (let's

say that the value is past 6000 (avg redline for passenger cars) -> we throw a warning and let's light up an LED.

- Now for this to work, we would need to work with the payload bits. For a value like 6000, we can put this into the payload using shifting. Now such a big number cannot fit in ONE payload element - so let's split it into two. 16 bits can get a max of 65535 > 6000. Lets say redline_RPM = 6000; payload[0] = redline_RPM >> 8 & 0xFF; //high byte. Payload[1] = redline_RPM & 0x00FF; //Low byte -> the payload now looks like 17,70,0,0,0,0,0,0 -> so in the IDS check, we extract the two numbers and then OR them together to get the hex value - compare this hex value with the redline.
- So that's clear but what I don't get is when and how the WRONG message will come through (do we set a period for it for testing purposes - or do we use an interrupt driven button??)
- Now let's discuss the headline where we talk about infrequent stuff. A message arriving too quickly or too late.
- And now that I think of it - we can def work with different tests (like we did for RTOS) to test the system - we don't need a button or anything like that - the same file with a few anomalies that can be detected and shown on the dashboard.
- So for timings, we can use last_Transmitted - let's say a message is supposed to arrive every 1 second - if from last transmitted to current timestamp, the difference is > 1, we know it's too late. And if the message fires early - we can work with the last-transmitted and current timestamp again
- For the unexpected IDs, it is a simple checker where for every transmitted message, if the canID != message[0] | message[1]…etc. we throw an error
- I feel that the detection should live in an ISR - safer for real-world applications
- Another per message stage would be last-arrival (I used last_transmitted above and that's wrong cuz we're only looking at arrivals - same technique and idea though)
- We can look at timestamps again - at 1 s, this message should have arrived but didn't - throw an error
- For an anomaly, we can just simply add a line in UART explaining the issue - we can use a switch case statement before the print and use a flag for the different cases for the different print statements as well.
- So for the anomaly detections - we need to catch it after the frame is transmitted and within the ISR - so the cases can be INSIDE the ISR - the message objects have all the elements we need so that can be a thing

- Ok so we had some flaws in the IDS design - the ISR handling all detection systems - but we overlooked the fact that a message never sent would never be received and hence would not show up in the ISR
- SOO, the ISR manages the 'value-out-of-range' and 'arrived too fast' problems -these exist on the receiving side so we can def put it in the ISR
- The deadline triggered one needs to be periodic and in main. We need to check if a message has never transmitted or received - it can be done using systick where we check every 1ms for the anomaly in main - and we need to add tolerance as well because sometimes messages might get delayed due to measurement delays or other issues not worth flagging and stopping the system for.
- For every message, we ask is now_time - last_ARRIVAL > period + tolerance
- So our design now has both halves.
- Also, the values being out of range is 'anomaly' and the frames arriving late/early/never is a signature mismatch

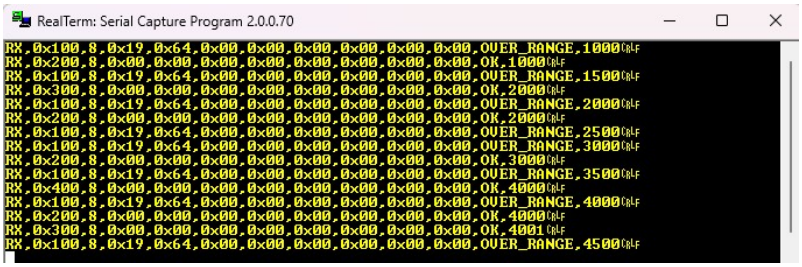
- Ok so for different IDs - should we add the promiscuous stuff of no?? Im lowkey thinking add it - there's no shortage on time - let's make sure the system is as close to the real deal as possible.
- So promiscuous is basically a broadcasting manner - Every ECU on the bus receives every transmitted message. BUT, nodes rely on acceptance filters to process only the arbitration IDs relevant to them.
- Too slow would be delta > period + margin
- Too fast would be delta < period - margin
- ISR -> check payload value ranges AND check if the message is too-fast.
- Main -> periodic deadline
- For the fault tests: **over-redline RPM value** (only checking the RPM - considering we're looping through the message array, we will have access to all of the events. **off-schedule frame** (all three cases - never arriving - too fast - too slow). Also, **wrong CANID.** This is something I can design after completing and testing the two above - this will require some register work.
- Also, all of this is ANOMALY detection - not signature stuff - Signature detection is matching against a database of known-bad patterns. So NO SIGNATURE PATTERN
- Anomaly: modelling normal and flagging deviations.

- Design:

Let's work with the ISR first:

- First, let's check the value ranges and validity. Let's do it for all of them and add them in the table setup.
- Good idea is to make an enum with different error types. Every message will have this 'status' which can help in running the UART prints in main.

- Ok for the error handling - im doing if (current message's payload >= 6000) -> error
- So for the max values, Ive set up a maxVal field in the message struct and using it to hardcode everyone's maxVal using the Table Setup - this prevents us from hardcoding in the ISR
- Extract the payload value using [0] and [1].
- Check
- Then make an else block which sets the status back to OK
- We make the current message's payload value to be 6500 (0x1964)
- Result:



All good!!!

Now let's work with main and the timing violations:

- So it's set in main
- We have a period for every element and that basically makes up for a sort of heart bear with a specific frequency
- Our job is to compare that beat with the one we're getting on the bus at this exact moment - so compare the EXPECTED one to the ACTUAL one.
- So too slow and missing can be under the same umbrella.
- We check if Delta (Delta = Now - lastArrived) < period - margin. If so, we know it is too fast. Make sure u do th math properly cuz the period+margin I did originally makes a valid message invalid - always trace your own code and methodology out please. That's our first check.
- Now, we need to figure out on how to get the proper

lastArrived value. So at first, we know that lastArrival would be garbage or zero. So, let's work with a new flag we can put in the struct which tracks if this is the first arrival or not (per object)

- Ok so if this flag is zero, the message has never been transmitted - so we just print it out, set the flag to 1, and set lastArrived to current time. Then clear the interrupt flag
- Now, we need to work with the else block - check if it is too fast - if so, set the status to too fast, print, set lastarrived to current time and clear interrupt flag
- Also, I just learned that clearing the interrupt flag is a read-write modify sequence. So, we must disable interrupts before and enable them after clearing the flag.

- Now for the too slow/missing message - we need a constant watchdog which consistently checks the timings and receivals to see if the message has arrived yet or not.
- Check if the arrivalFlag is 1 -> if so, find delta which is the difference between now and lastArrival.
- If (delta > period + margin) -> set status to MISSING and print - then set the flag back to ZERO

- Ok so one lesson here -im trying to play with the lastArrived and period for timing fault injections - but the issue is that we do not want to play with the code - we want to inject faults into the bus - so we play from the TX side and use lastTransmitted as well. Also, period is being used for both transmission and the catching of these faults - so we can’t change it.
- SOOO, send a frame right after the original send
- Also, make the fault injection reproducible. At 'this second' send another frame (an extra one) - this should be caught by your system.
- Let's design it: Well I initially thought to add in a second transmit in the main function but it didn’t quite work. I added a for loop in between which was lesser than the period of the message and sent the message again after the loop ended so that it was sent before it was supposed to be sent - well, that didn’t work out - I checked the UART output and all it changed were the timestamps -so that failed as well - then I realised that ticks kept going up despite the for loop which is probably why it didn’t work - but then I put it in a disabled interrupt zone - that didn’t work either - no clue why
- Now, im looking into a different way - AI advised to think like an attacker of the bus rather than the engineer trying to break his own

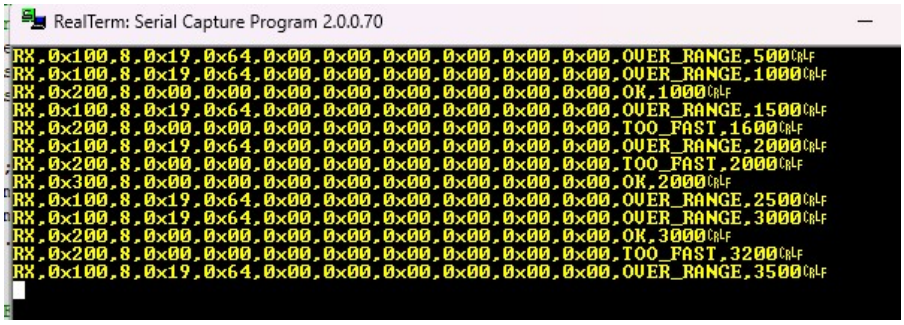
code to look for faults - so let me write up a piece of code in the main function which SENDS in the wrong stuff to the BUS

- ok this is my first thought:
  //THIS IS WHERE THE ATTACKER WORKS
  if (ticks - attackerTime >= 100) {
  CAN0_Transmit(message, i);
  attackerTime += message[i].period;
  }
  this exists in the main for loop - right next the actual transmit
  ok this didnt work because it flooded everything - none of them were too fast though - let me do a controlled test specific to throttle.

- Throttle is Index 1. Period of 1000ms. Margin of 100ms. Let's keep the extra AttackerTime variable. Lets say that the attack happens every 1600ms. Look at my code
- //THIS IS WHERE THE ATTACKER WORKS
  if (ticks - attackerTime >= 1600) {
  CAN0_Transmit(message, 1);
  attackerTime += 1600;
  }

- WELL, IT WORKSSSS

- One thing im concerned about is that it called a proper send TOO FAST if the previous too fast was a little before it - look at the pic below



- The one at 2000ms is also TOO FAST - we can make a specialised check that makes anything at an exact period OK, but that can collide with other checks like over_range and also there is a jitter for 0x400 because all 4 messages are coming in at once. They come in order as well due to ID priorities. SO, the 0x400 comes in at a 1ms delay than it is supposed to - so let's not add that hard-coded line
- Also, we added the attacker in the loop just for better detail and more sample points for better decision-making regarding next

steps or efficiency of the system

- Ok all good - we need to hurry now - let's complete the MISSING and python dashboard stuff now.
- MISSING: Idea: Arbitration. I tried something dumb - removing CAN0_Transmit from the main if loop which transmits everyone on their periods. BUT, left the ATTACKER loop which transmitted THROTTLE every 1600ms. So, I saw that the ticks kept going up, and on every few transmissions, the next one came out to be MISSING. So, it technically works, but we need to test it in a more efficient method.
- Wait - what if I add in something that just wastes ticks without transmitting anything - that way, the next time something comes in, the delta is way bigger than the period + margin that it is deemed as MISSING/too slow - nope doesn't work
- Let's pick up a victim for this lol - Vehicle Speed, period 2000ms.
- Ok we're overcomplicating it - let's just make it so that after a certain number of ticks, we do not transmit a specific event.
- Let's do it in main in the while(1) loop but don't touch the other stuff
- Yeah not quite getting it here - how do I essentially STOP something from transmitting. Block a function? How though?
- Oooh, we can also stop it from being RECEIVED - so something like disabling interrupts - that way, we don't do CAN_IRQ and it might work - let's test

```
//THIS IS WHERE THE ATTACKER WORKS FOR MISSING
if (ticks - attackerTime_M >= 3000) {
	__disable_irq();
	CAN0_Transmit(message, 2);
	__enable_irq();
}
```

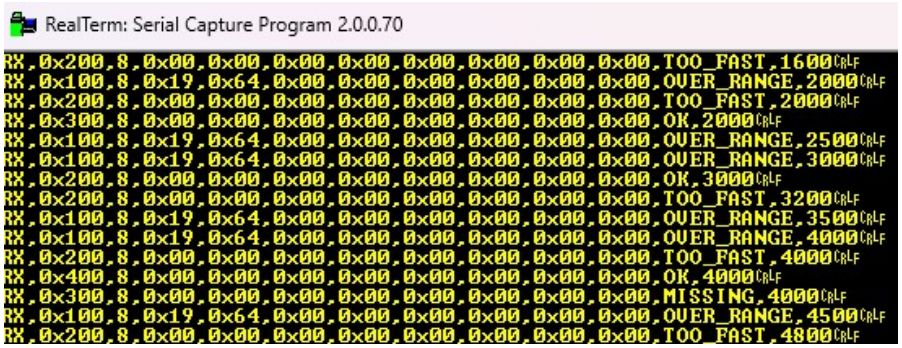
- Yep didn't work - instead starting FILLING THE WHOLE thing with messages
- Ok same method of interrupts, but different approach.
- Speed comes in 2000ms - let's make it miss the 4000ms deadline
- Ok did something bad - made a new function - using abstraction I think - saw it on the internet - some guy went into the hardware function and made it return; if the conditions werent met - I tested it with the actual transmit function - it works to some extent - it catches that the object is missing from its next transmission, but gives the wrong timestamp - the old timestamp where It first came up.

```
void CAN0_Transmit(volatile Msg* message, uint32_t index) {

		if ((ticks >= 2500) && (ticks <= 4500) && (index == 2)) {
			return;
		}
```

```
Else {
        //REST OF THE FUNCTION
}
```

- Ok need help here - is this the right way or not.
- Yeah this works - just bringing in more abstraction right now
- Ok done - heres the final result

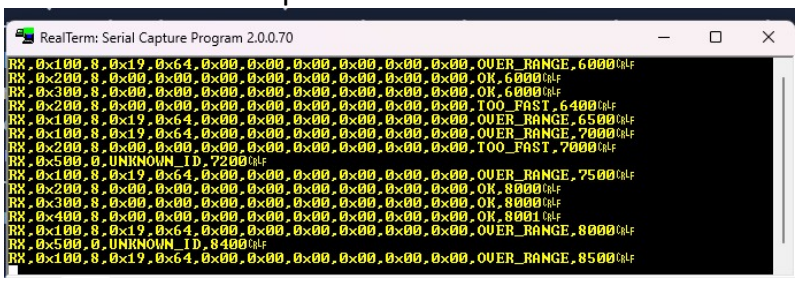


- It works guys less gooooo
- It shows that the message was missing at 4000ms where it was supposed to land

- Abstraction works as well - let's do promiscuous now. Identifying unknown IDs and flagging them.
- So for this, we need to do acceptance filtering. We want to configure a specific message object to only accept messages that match a particular identifier.
- So, the basic setup is the same - but, we need to do something with the MASK bit on the CMSK register and see the MASK stuff on the others as well - ARB/MCTL
- For CMSK, set the MASK bit so the transfer also writes the mask registers into the object.
- ARB2 - Set MSGVAL to show that the message is valid. We don’t need the ID value here - the Masks make it don’t-care.
- Now we add another Register - the CAN_IF2MSK2 register. This MSK, MDIR, and MXTD. These are used for acceptance filtering.
- We can set the MSK bit so that the corresponding ID is used for acceptance filtering. I don’t think we need Mask Extended Identifier (MXTD). And DIR, ummm I don’t think we need it - but we can set it so we know that if it is COMING -wait, a little correction on my thing here - im trying to accept everything - so make it don’t care - so we clear the MSK bit to 0. and leave the rest
- Now, work with the MCTL register - keep everything the same, then set UMASK to use mask for acceptance filtering
- Now, write object number 5 using CRQ register then poll until BUSY clears
- I made a function which transmits an ID (0x500) every 1200ms. It is called once at the start of our main.c while loop. The check is done

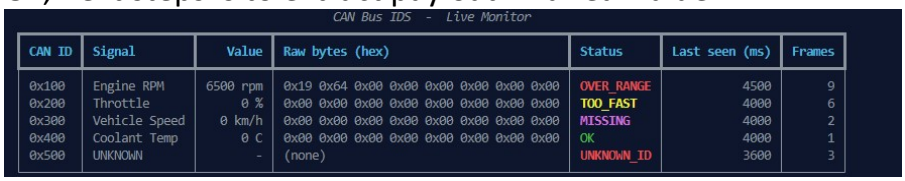
outside the for loop in main by checking if the interrupt flag was set for object 5. The ISR checks if the object number is 5. If so, it sets the status

- It works. Heres the pic
-

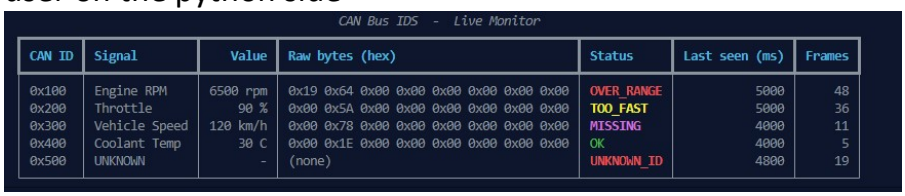


ONTO THE DASHBOARD

- We need pyserial and rich
- Pyserial is used to enable serial communication between the PC and device
- Rich is used for rich text and formatting in the terminal. Makes a good Text user interface (TUI)
- In order to read, use a with statement and use the .Serial function from the serial library. Give it the PORT, Baudrate, and timeout=1, and loop through by using serial.readline() function. Then, print using print(line.decode('utf-8')). This is new syntax for python and it means converting bytes into standard string
- YAYY - got it working
- It's reading like realterm now.
- Ok, next steps is to extract payload in a real value.
-

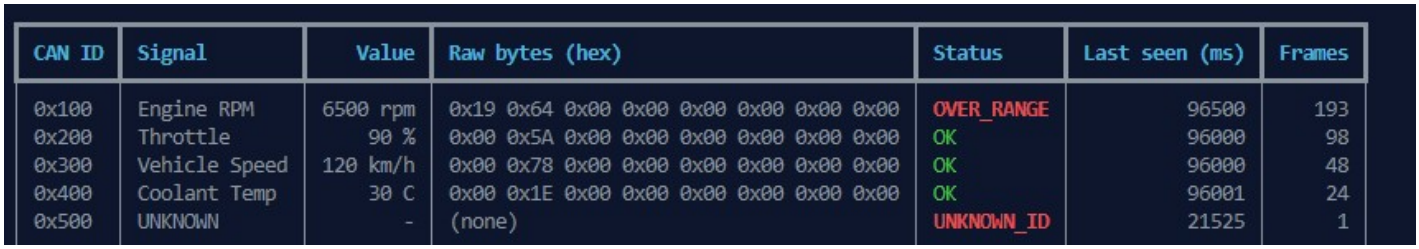


- DONEEE
- Ok now let's test with some actual values for the pic, then make the firmware side responsible for taking in Injection frames from the user on the python side
-



- Ok now onto the buttons lol

- So we need the system to take in inputs through the UART setup of the MCU

- We can enable UART RX.
- 

CAN ID	Signal	Value	Raw bytes (hex)	Status	Last seen (ms)	Frames
0x100	Engine RPM	6500 rpm	0x19 0x64 0x00 0x00 0x00 0x00 0x00 0x00	OVER_RANGE	96500	193
0x200	Throttle	90 %	0x00 0x5A 0x00 0x00 0x00 0x00 0x00 0x00	OK	96000	98
0x300	Vehicle Speed	120 km/h	0x00 0x78 0x00 0x00 0x00 0x00 0x00 0x00	OK	96000	48
0x400	Coolant Temp	30 C	0x00 0x1E 0x00 0x00 0x00 0x00 0x00 0x00	OK	96001	24
0x500	UNKNOWN	-	(none)	UNKNOWN_ID	21525	1

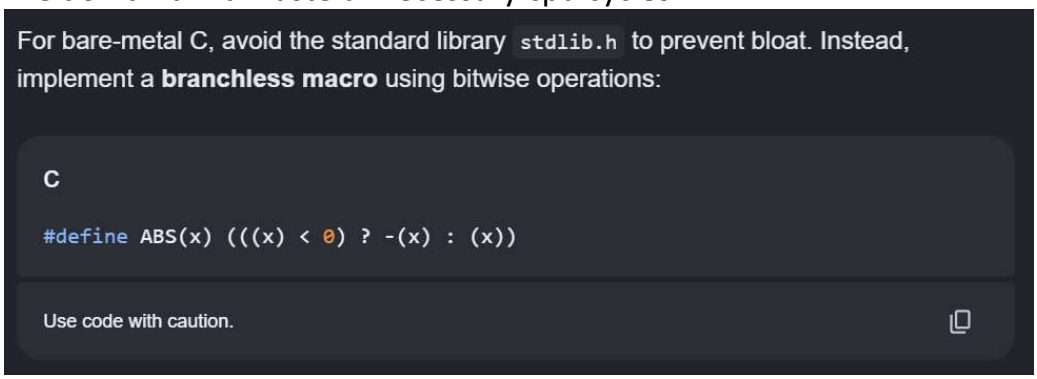
- Ok we doneee - it was rather simple to do the RX part - just made the input Uart function non-blocking, used a flag, and set up a switch case statement to call each function separately
- F -> too_fast injected on THROTTLE
- U -> unknown ID injected
- O -> over-range injected on RPM
- So user part is done
- Verified in both realterm and python dashboard - will upload realterm videos to the notion for this specific part as we can see more clearly which user inputs are going in using UART RX

- Onto the next feature - catching spikes or the odd rate-of-change of values. Kinda easy - look at the documentation for detail

New C syntax learned:

- There is more theoretical C I learned which is included in the mistakes or general notes section
- \r in printing statements (used in UART_printf) is a carriage return output cursor is moved to the beginning of the current line without moving down to the new line.
- For UART, we can print out straight string (chars) using the UART_printf function we have which uses UART_outchar (prints out char by char) in a loop. However, when we are trying to print out integers, hex values, etc, we cant just pass this into the printf function. This is why we have a print_buffer array of size 1023. We use **sprintf** which is basically for strings and writes directly into the buffer array we have. Let's see the flow for that:
  sprintf(print_buffer, "My age is (%d) \r\n", number);
  UART_printf(print_buffer);
- Sprintf saves the stringed out version to the print_buffer which is then easily printed using the basic uart printf function.
- This is more of a theoretical lesson - and a repeated mistake from classwork - first time in building something so worth noting down so as to not repeat again -> in C, an array and a pointer are not

exactly the same thing, but they act like it when passed to functions. When you use an array's name by itself (like messageTable), it 'decays' into a pointer to its very first element. Because of this rule, if a function is expecting a pointer (Msg* msg), we can simply pass it the array's name. I got confused because I was dealing with an array of structs and had to populate each struct in the array. This is clear now in terms of using only the array name in the parameter even when the function asks for a pointer, as they are essentially passed in the same way. Passing an array is the same as passing a pointer to its first element.

- So when defining a preprocessor constant, we can add suffixes to make it U or UL (unsigned 32-bit) - main thing is though I was trying to make these constants for the error values and thresholds - I learned that the preprocessor constant will default to a SIGNED 32-bit integer - so if we need a different var type, we must write it out. We don't wanna waste unnecessary cpu cycles
- 

- Lets say x=-5, -5<0, -(-5) = 5 +5
-

Some New stuff I learned (a lot learning moments in the overall notes but I included the ones I remembered to add here) (mistakes also included here):

- With struct, we should pass a pointer to it when using it in a function. The reason for this is because if we use pass-by-value, (just passing a normal struct) the entire data must be copied - the struct can be huge and this can cause performance issues. Therefore, we use pass-by-reference. But remember we might not want the function to change the struct fields which is a possibility when using pass-by-reference. Therefore, end up using const struct* data as a parameter when passing through a function. This ensures nothing can be changed in the original data.
- I learned about data size limits when doing some allocations - for instance, the payload index is uint8_t but my pattern was allocating a number to it >255 which meant it would loop back to 0 and be a problem. Make sure we know the limits and stay under them.

- So I was confusing priorities when writing the continuous RX system. CAN bus has ID priorities, where the lowest Message object (1) has a higher priority than the highest message object (32) (total of 32 objects can be sent and configured on my system). THIS IS KNOWN AS THE **Bus Arbitration Priority.** It only matters when there are multiple independent nodes contending for the bus at the same time - we only have two nodes.
- Ok so when using the NVIC_SetPriority function from msp432e401y.h, it needs two parameters. One is an interrupt number, in our case called CAN1_IRQn. The other is priority. 0xFF is the lowest priority.
- Go in a flow - Set bit 1 of CANCTL register to enable CAN interrupts. Use NVIC_SetPriority. Then, use NVIC_EnableIRQ by providing it with CAN1_IRQn and we're good. For this, the second priority parameter doesn't matter and can be left empty.
- Ok so I just learned a bit more about interrupts and volatile in general. We defined the message object in can.h using both the extern and volatile keywords - the extern allows it to be used within can.c and main.c and anywhere else -as we know, it tells the compiler that this exists in other files where it is defined - declared already here - volatile allows the hardware (our ISR register writes) to keep the message object constant across sends and files.
- Just learned something pretty cool about using numbers like bitmasks. In my ISR, I was setting a flag to 1 from 0. Then, I also needed other information like the object number from the ISR but we cant return anything - so, we can use the flag as a bitmask - like a register - Bit 0 is object 1 and so on. Flag value stays > 0 and we keep running the loop without issues - the main.c file gets the flag, and can extract which bit is set to get the object number which becomes the index for printing.

- Ok so I just designed the IDS system's anomaly checker - I hardcoded each value for the max values for every event. The better approach is to add a maxValue element to the message struct which can be set in the table setup - which is totally fine for hardcoding - this keeps the scalability.

- Ok so my tolerance for the THROTTLE field was 0 which was causing a false 'error' on the bus - it thought it was too fast. This happens due to a jitter in the timing values - might be 999 instead of a 1000 which can mess up the status. So remember that adding tolerance is important.
- Ok so while doing the abstraction stuff for attacks, I passed in the exact value of attackerTime which existed in main into a functino

which was supposed to change its value. But because this was pass-by-value, the number didn't change - make sure to pass in a pointer to this integer variable. Call it using & as we know.

Testing logs:

1. First test to be done was to check if my CAN transmit is working properly. Based on the debugger, the STS register's TXOK bit came out to be 0. This meant it never transmitted anything. Every other register was normal and working as expected. Another thing was the ERR register not having any transmission errors. This meant that it neither succeeded nor failed. Therefore, we had to test it properly using a loopback system where the TX is also the RX. The method is described above in the doc. The code is shown below:

```
void CAN0_Init(void) {
    //RCGCCAN Register setup. It is used to let
software enable and disable the clock to the CAN
module
    SYSCTL_RCGCCAN_R |= SYSCTL_RCGCCAN_R0;
//enables clock for CAN0
    while ((SYSCTL_PRCAN_R&SYSCTL_PRCAN_R0) == 0)
{} //waits until bit 0 is set
    CAN0_CTL_R |= CAN_CTL_INIT; //INIT bit set to
1
    CAN0_CTL_R |= CAN_CTL_CCE; //CCE bit set to 1
    //Calculated values for the CANBIT Register
    uint32_t brp = 14; //14
    uint32_t sjw = 1 - 1; //0
    uint32_t tseg1 = 12 - 1; //11
    uint32_t tseg2 = 3 - 1; //2
    //CAN0_BIT_R is the CAN Bit Timing Register.
[0:5] is the BRP field, [6:7] is the SJW field,
[8:11] is the TSEG1 field, and [12:14] is the
TSEG2 field
    CAN0_BIT_R = (brp << CAN_BIT_BRP_S) | (sjw
<< CAN_BIT_SJW_S) | (tseg1 << CAN_BIT_TSEG1_S) |
(tseg2 << CAN_BIT_TSEG2_S);
    //CAN0_BIT_R = (brp << 0) || (sjw << 6) ||
(tseg1 << 8) || (tseg2 << 12);

    //Add in Loopback teting
    CAN0_CTL_R |= CAN_CTL_TEST; //set the test bit
in the control register to unlock the TEST
register which was previously read-only
    CAN0_TST_R |= CAN_TST_LBACK; //enable LBACK
(internal loopback)
```

```
    CAN0_TST_R |= CAN_TST_SILENT; //enable SILENT
so that nothing leaks onto the real bus.
    CAN0_CTL_R &= ~CAN_CTL_CCE; //Clear CCE bit
    CAN0_CTL_R &= ~CAN_CTL_INIT; //Clear INIT bit
to leave initialisation stage

    LED1_ON(); //Turn on LED1 to indicate that the
initialisation was a success

}
```

Result: TXOK bit in the STS (status) register comes out to be 1. This means that the transmission was a success. Therefore, we can narrow down our issue to the wiring or RX which is not working properly.

After hours spent debugging hardware using the AD3, it was a wiring issue at first. The pin holes were not connecting properly to the pin headers. Therefore, I went back to the initial way of bending the wires to make electrical connection. I tested this using a new GPIO_TEST() function which set up both Port A and Port B as outputs - they sent out a HIGH signal in a specific frequency. I connected this to my AD3 scope and confirmed that the new wiring setup works perfectly. I could see the waves on my scope. Then, I connected the CAN system again, and ran the debugger, ran the program, and the CAN0 STS register's TXOK was set. CAN1 STS register's RXOK was set. This meant the transmission was successful. Next steps would be to check if the EXACT message was sent. A short interview story tbh. Not the whole lesson cuz the mistake was kinda odd and shows me as a lousy engineer, but the testing part was kinda fun so we can talk about that.

The code for the Pin test can be seen below:

```
void GPIO_TEST(void) {
  SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R1; //Enables
clock for Port b
  while ((SYSCTL_PRGPIO_R&SYSCTL_PRGPIO_R1) == 0) {}
//waits until bit 0 is set
    GPIO_PORTB_DIR_R |= 0x03; //Enables the first and
second bits to be outputs
    GPIO_PORTB_DEN_R |= 0x03; //Enables Digital I/O
on Port b

    while(1) {
        GPIO_PORTB_DATA_R |= 0x03;
```

```
            for (volatile int i = 0; i < 20000; i++) {}
            GPIO_PORTB_DATA_R &= ~0x03;
            for (volatile int j = 0; j < 20000; j++) {}
        }
    }
```

PRIME: Purpose - Research - Interview - Mechanics - Examples